I-Logical Approaches Logical Inference and Efficient Contractions

Foundations of Neuro-Symbolic AI

Alex Goessmann

University of Applied Science Würzburg-Schweinfurt (THWS)

April 20, 2026

Motivation: How to avoid the curse of dimensionality in reasoning?

Tensor network decompositions for efficient representation:

- Knowledge bases as tensor networks of formulas
- Formulas decomposed based on their syntax

Tensor network contractions for reasoning:

- Deciding entailment requires contractions
- Naive contraction approaches instantiate large tensors
- Curse of dimensionality prevents efficient and generic reasoning

Efficient reasoning with tensor networks

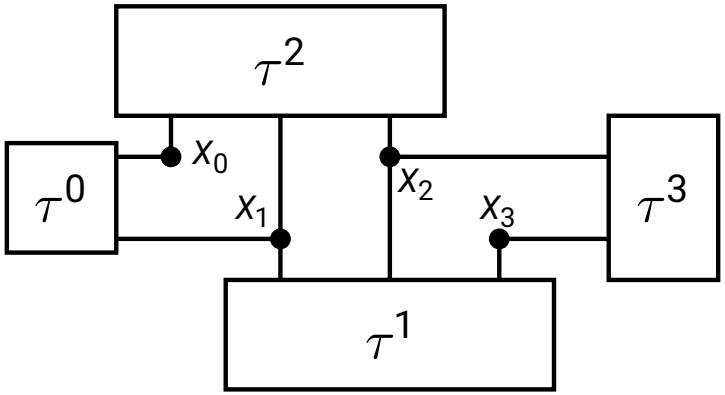
We investigate means to overcome the curse of dimensionality and find classes of efficient reasoning tasks.

Outline

- Feasibility of contractions
- Monotonicity of Entailment
- Modus ponens and forward chaining
- Constructing inference algorithms

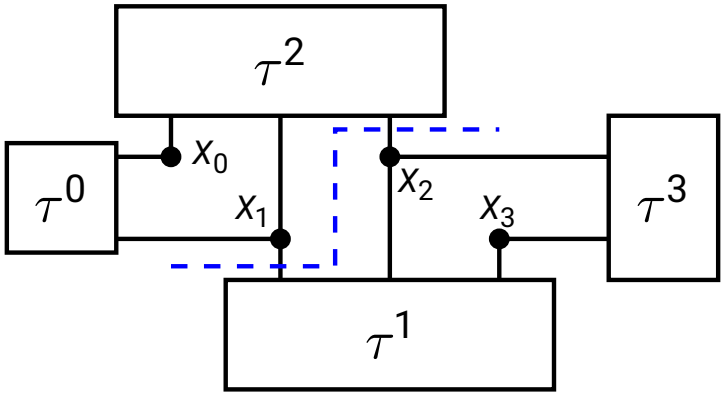
Contraction by sub-contractions

Tensor network contractions can be performed by orchestrations of sub-contractions:



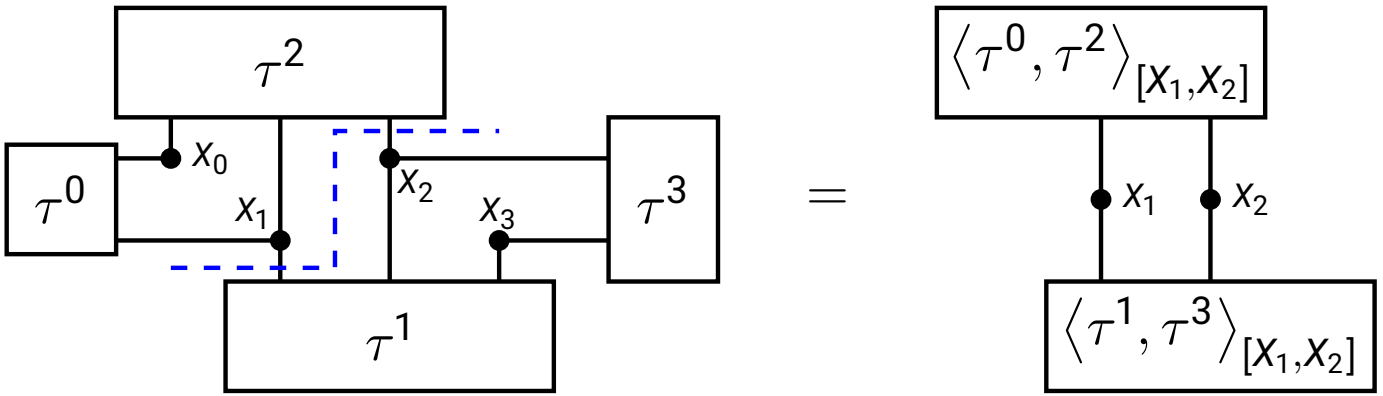
Contraction by sub-contractions

Tensor network contractions can be performed by orchestrations of sub-contractions:



Contraction by sub-contractions

Tensor network contractions can be performed by orchestrations of sub-contractions:



Efficiency advantage:

- Processing of tensors with all four variables X_0, X_1, X_2, X_3 can be avoided
- Sub-contractions $\langle \tau^0, \tau^2 \rangle_{[X_1,X_2]}$, $\langle \tau^1, \tau^3 \rangle_{[X_1,X_2]}$ are tensors of smaller orders
- For larger tensor networks with low connectivity this efficiency advantage is critical

The diagrammatic notation pays off

Tensor network diagrams avoid notation overload by sum and product symbols:

$$\left\langle \tau^0 [X_0, X_1], \tau^1 [X_1, X_2, X_3], \tau^2 [X_0, X_1, X_2], \tau^3 [X_2, X_3] \right\rangle_{[\varnothing]}$$
$$= \sum_{x_0 \in [m_0]} \sum_{x_1 \in [m_1]} \sum_{x_2 \in [m_2]} \sum_{x_3 \in [m_3]}$$
$$\tau^0 [X_0 = x_0, X_1 = x_1] \cdot \tau^1 [X_1 = x_1, X_2 = x_2, X_3 = x_3] \cdot$$
$$\tau^2 [X_0 = x_0, X_1 = x_1, X_2 = x_2] \cdot \tau^3 [X_2 = x_2, X_3 = x_3]$$

These transforms are visualized by cuts of the tensor network diagrams!

The diagrammatic notation pays off

Tensor network diagrams avoid notation overload by sum and product symbols:

$$\left\langle \tau^0\left[X_0, X_1\right], \tau^1\left[X_1, X_2, X_3\right], \tau^2\left[X_0, X_1, X_2\right], \tau^3\left[X_2, X_3\right] \right\rangle_{[\varnothing]}$$
$$= \sum_{x_1 \in [m_1]} \sum_{x_2 \in [m_2]}$$
$$\left(\sum_{x_0 \in [m_0]} \tau^0\left[X_0 = x_0, X_1 = x_1\right] \cdot \tau^2\left[X_0 = x_0, X_1 = x_1, X_2 = x_2\right] \right) \cdot$$
$$\left(\sum_{x_3 \in [m_3]} \tau^1\left[X_1 = x_1, X_2 = x_2, X_3 = x_3\right] \cdot \tau^3\left[X_2 = x_2, X_3 = x_3\right] \right)$$

These transforms are visualized by cuts of the tensor network diagrams!

Outline

- Feasibility of contractions
- Monotonicity of Entailment
- Modus ponens and forward chaining
- Constructing inference algorithms

Motivation

Contractions of large knowledge bases get infeasible.

Is it always necessary to perform large contractions?

Example: Entailment in the toy accounting system

When knowing that feature X_F is true, we can conclude that X_{A1} is true, without considering other accounting rules.

The efficiency-generality tradeoff

A fundamental tradeoff underlies all reasoning schemes:

Generality

- Which reasoning tasks can be answered by a specific scheme?
- Most general: Entailment given any knowledge bases
- Here: Generic reasoning by **global** contractions

Efficiency

- Which computational resources are necessary for a reasoning scheme?
- Here: Efficient contractions by orchestration of **local** contractions

Example: Entailment in Horn logics can be decided efficiently.

Monotonicity of Entailment

Theorem (Monotonicity of Entailment)

Given a knowledge base $\mathcal{KB}$ of formulas, let $\tilde{\mathcal{KB}} \subset \mathcal{KB}$ be a subset and let f be another propositional formula. If $\tilde{\mathcal{KB}} \models f$ then also $\mathcal{KB} \models f$.

Proof sketch: If $\tilde{\mathcal{KB}} \models f$ then

$$\left\langle \tilde{\mathcal{KB}}[X_{[d]}], \neg f\left[X_{[d]}\right]\right\rangle_{[X_{[d]}]} = 0\left[X_{[d]}\right]$$

and since this is a sub-contraction of $\left\langle \mathcal{KB}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{[\varnothing]}$ also

$$\left\langle \mathcal{KB}\left[X_{[d]}\right], \neg f\left[X_{[d]}\right]\right\rangle_{[X_{[d]}]} = 0\left[X_{[d]}\right]$$

and therefore $\mathcal{KB} \models f$.
Notice that the back direction does not hold: When $\mathcal{KB} \models f$, then $\tilde{\mathcal{KB}} \models f$ does not always hold.

Invariance of adding entailled formulas

If $\mathcal{KB} \models f$, then for any $x_{[d]} \in [2]^p$

$$\mathcal{KB}\left[X_{[d]} = x_{[d]}\right] = \begin{cases} 1 & \text{if } x_{[d]} \text{ is a model of } \mathcal{KB} \\ 0 & \text{else} \end{cases}$$

$$= \begin{cases} \mathcal{KB}\left[X_{[d]} = x_{[d]}\right] \cdot f\left[X_{[d]} = x_{[d]}\right] & (\text{since } \mathcal{KB} \models f) \\ \mathcal{KB}\left[X_{[d]} = x_{[d]}\right] \cdot f\left[X_{[d]} = x_{[d]}\right] & (\text{since } \mathcal{KB}\left[X_{[d]} = x_{[d]}\right] = 0) \end{cases} .$$

Consequence: When $\mathcal{KB} \models f$, then

$$\mathcal{KB}\left[X_{[d]}\right] = \left\langle \mathcal{KB}\left[X_{[d]}\right], f\left[X_{[d]}\right] \right\rangle_{[X_{[d]}]} .$$

Interpretation: Memory of entailed formulas

We can add entailed formulas to a knowledge base.

Usage for efficient reasoning

Efficiency: If $\tilde{\mathcal{KB}}$ is a small subset, $\tilde{\mathcal{KB}} \models f$ can be decided efficiently.
Reuse: Entailed formulas f can be added to the knowledge base and used in further $\tilde{\mathcal{KB}}$ choices.

- Messages: Entailed formulas f "summarize" subsets and are added to knowledge bases
- Propagation: Entailed formulas are propagated throughout the knowledge base and

Design of inference algorithms

Logical inference algorithms are designed based on the monotonicity and memory properties:

- Choice of pairs $\tilde{\mathcal{KB}}, f$ such that $\tilde{\mathcal{KB}} \models f$: Often syntactical rules are applied
- Priority of adding f to $\mathcal{KB}$: Need to apply restrictions to avoid overloads

Outline

- Feasibility of contractions
- Monotonicity of Entailment
- Modus ponens and forward chaining
- Constructing inference algorithms

Inference using Syntax

So far: Inference and entailment is formulated by semantics (with tensor interpretations).

Now: Syntactical Inference Rules find entailed formulas using syntactical representations only.

Notation: When $\tilde{\mathcal{KB}} \models f$ we denote

$$\frac{\overbrace{\tilde{\mathcal{KB}}}^{\text{has to be known for the rule to apply}}}{\underbrace{f}_{\text{is added to the knowledge base}}}$$

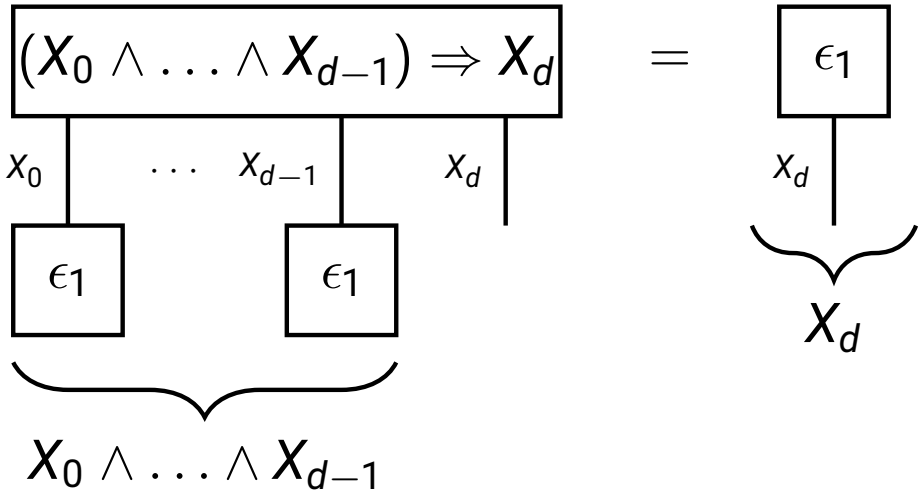
Modus Ponens

Modus Ponens Inference Rule

Modus Ponens is the simplest inference rule: When an implication holds and the premise is true, then the head follows.

$$\frac{(X_0 \wedge \ldots \wedge X_{d-1}) \rightarrow X_d\,,\ (X_0 \wedge \ldots \wedge X_{d-1})}{X_d}$$

Modus ponens corresponds with the contraction equation:



Clauses

Recall: Given $\mathcal{U} \subset [d]$ and $y_{\mathcal{U}} \in \times_{k \in \mathcal{U}}[2]$ the corresponding clause is

$$Z^{\vee}_{(\mathcal{U}, y_{\mathcal{U}})}[X_{\mathcal{U}}] = \mathbb{I}[X_{\mathcal{U}}] - \epsilon_{y_{\mathcal{U}}}[X_{\mathcal{U}}]$$

Special types:

- Horn clause: There is at most one $k \in \mathcal{U}$ with $y_k = 1$
- Definite clause: There is at exactly one $k \in \mathcal{U}$ with $y_k = 1$

Interpretation of definite clauses: Implications

Example: If in addition to F being present on an invoice, an invoice came from buisness partner $BP1$, then its booked using account $A1$.

$$\begin{aligned}\left(X_F \wedge X_{BP1} \rightarrow X_{A1}\right) &\Leftrightarrow \neg\left((X_F \wedge X_{BP1}) \vee X_{A1}\right) \\ &\Leftrightarrow \left(\neg X_F \vee \neg X_{BP1} \vee X_{A1}\right)\end{aligned}$$

Iteration of modus ponens inference rule

Given: A knowledge base of implications (definite clauses) in the form

$$\Big(\bigwedge_{k \in \mathcal{M}} X_k \Big) \to X_{\tilde{k}}$$

which are clauses

$$\Big(\bigvee_{k \in \mathcal{M}} \neg X_k \Big) \vee X_{\tilde{k}} \quad \text{parametrized by} \quad (\mathcal{U}, y_{\mathcal{U}}) = \big(\mathcal{M} \cup \{\tilde{k}\}, (1_{\mathcal{M}}, 0)\big)\,.$$

We iterate:

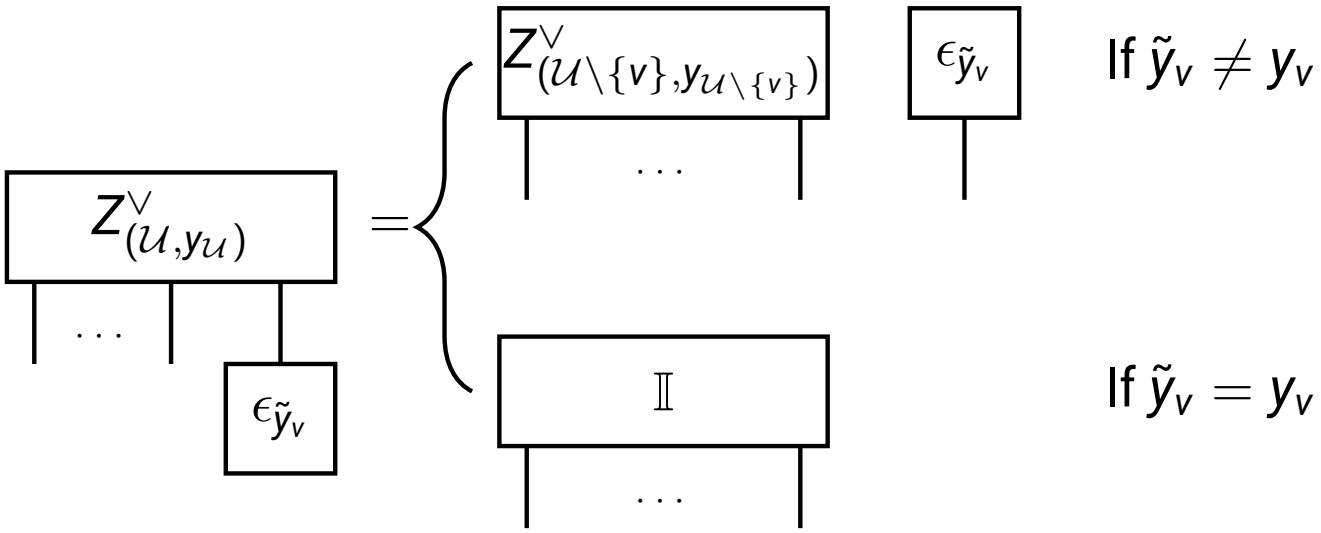
- Computation of **messages**: The formula X_d is entailed by the local knowledge base

$$\tilde{\mathcal{KB}} = \Big\{ \Big(\bigwedge_{k \in \mathcal{M}} X_k \Big) \to X_{\tilde{k}} \Big\} \cup \Big\{ X_k \,:\, k \in \mathcal{M} \Big\}$$

- The message $X_{\tilde{k}}$ is **propagated** and used for further local inferences.

More general: Contraction of clauses

For generic clauses we have



Therefore:

- When clauses "disagreeing" on v: Larger clause shrinks
- When clauses "agreeing" on v: Larger clause becomes trivial

Forward Chaining

We can iteratively simplify clauses $Z^{\vee}_{(\mathcal{U},y_{\mathcal{U}})}[X_{\mathcal{U}}]$ using the unit clauses $\epsilon_{\tilde{y}_v}[X_v]$.

Forward Chaining Algorithm

Require: Set of clauses $\mathcal{C}$ (each parameterized by $(\mathcal{U}, y_{\mathcal{U}})$)
Ensure: Entailed literals $\mathcal{L}$, remaining set of clauses

Initialize $\mathcal{L} = \varnothing$
while exists a clause $(\{v\}, \tilde{y}_v)$ of length 1 in $\mathcal{C}$ **do**
  for all clauses $(\mathcal{U}, y_{\mathcal{U}})$ with $v \in \mathcal{U}$ **do**
    if $y_v \neq \tilde{y}_v$ **then** (then the clause shrinks)
      $(\mathcal{U}, y_{\mathcal{U}}) \leftarrow (\mathcal{U} \backslash \{v\}, y_{\mathcal{U} \backslash \{v\}})$
    else (then the clause becomes trivial)
      $\mathcal{C}.\mathrm{drop}((\mathcal{U}, y_{\mathcal{U}}))$
    end if
  end for
  $\mathcal{L}.\mathrm{append}(\{v\}, \tilde{y}_v)$ (the literal is entailed and added to solution)
end while

Knowledge base of clauses

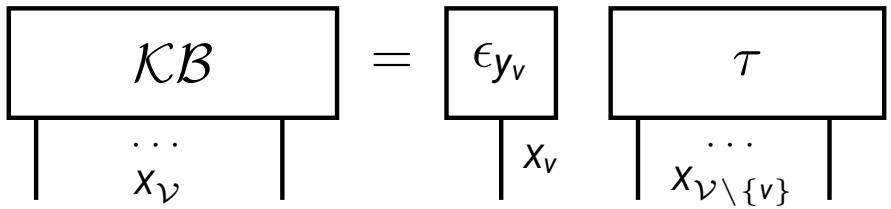
The knowledge base is a tensor network

$$\mathcal{KB}\left[X_{\mathcal{V}}\right] = \left\langle Z^{\vee}_{(\mathcal{U},y_{\mathcal{U}})}\left[X_{\mathcal{U}}\right] \,:\, (\mathcal{U},y_{\mathcal{U}}) \in \mathcal{C} \right\rangle_{[X_{\mathcal{V}}]} .$$

Notice: We have $\mathcal{KB} \models Z^{\vee}_{\{v\},y_v}$ if

$$\begin{aligned}\mathcal{KB}\left[X_{\mathcal{V}}\right] &= \left\langle \mathcal{KB}\left[X_{\mathcal{V}}\right], \epsilon_{y_v}\left[X_v\right]\right\rangle_{[X_{\mathcal{V}}]} \\ &= \epsilon_{y_v}\left[X_v\right] \otimes \left\langle \mathcal{KB}\left[X_{\mathcal{V}}\right], \epsilon_{y_v}\left[X_v\right]\right\rangle_{[X_{\mathcal{V}\setminus\{v\}}]} .\end{aligned}$$

To decide entailment of a literal we are searching for decompositions as



Literal entailment in knowledge bases

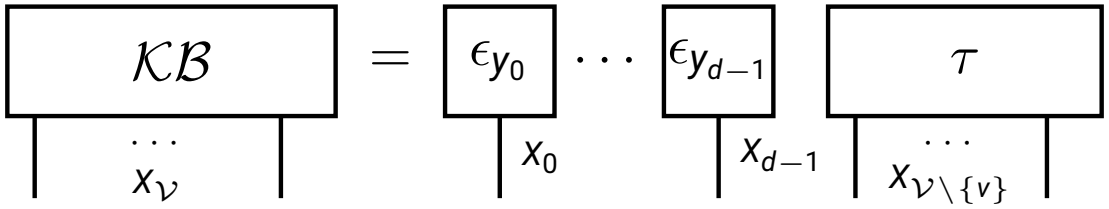
Reasoning task

We want to decide, which literals $Z^{\vee}_{\{v\},y_v}$ are entailed by a knowledge base.

We are searching for a maximal set $\mathcal{W}$ such that for a tuple $y_{\mathcal{W}}$ and some tensor $\tau\left[X_{\mathcal{V}\setminus\mathcal{W}}\right]$

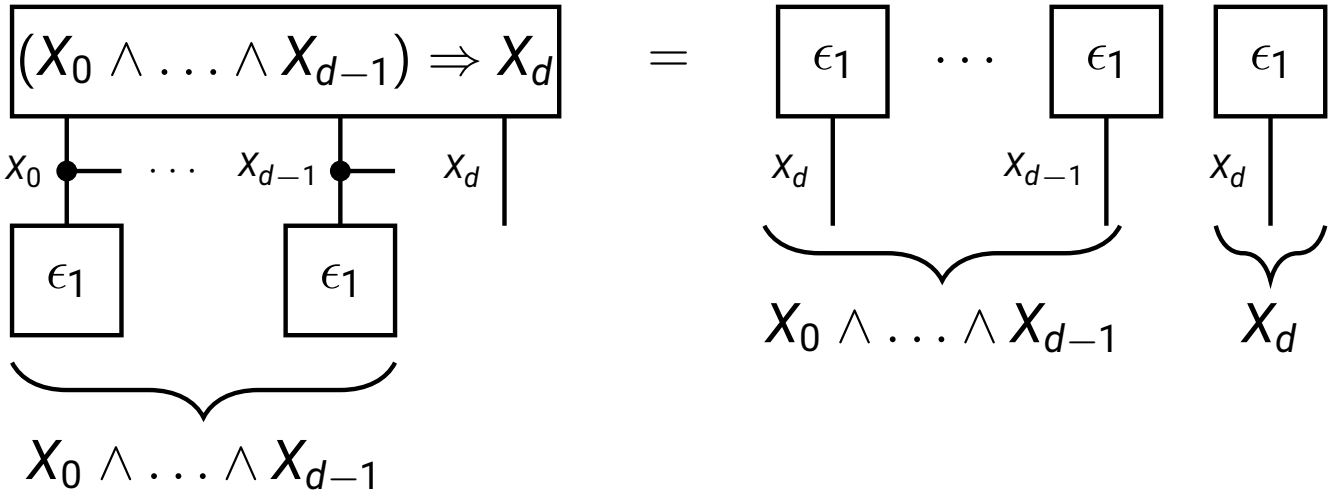
$$\mathcal{KB}\left[X_{\mathcal{V}}\right] = \left(\bigtimes_{v\in\mathcal{W}} \epsilon_{y_v}\left[X_v\right]\right) \otimes \tau\left[X_{\mathcal{V}\setminus\mathcal{W}}\right] .$$

This is a decomposition (when enumerating $\mathcal{W}$ by $[d]$)



Modus ponens as a sub-contraction

Modus ponens is disentangling the tensor network:



Understand modus ponens as a sub-contraction:

- Separate from the knowledge base $\mathcal{KB}$ the clauses

$$Z^{\vee}_{\left([d], 0_{[d-1]} \times 1\right)} \quad \text{and for } k \in [d-1] \text{ the literals} \quad Z^{\vee}_{\left(\{k\}, 0\right)}$$

- Contract the separated clauses
- Split the contraction into a tensor product of ϵ_1 vectors

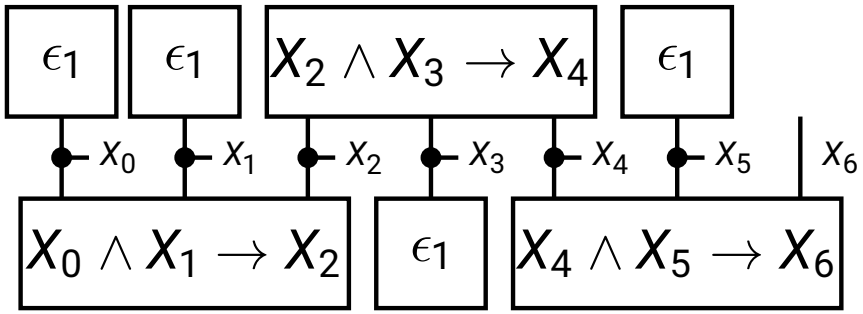
Forward chaining: Disentangling the network

Forward chaining as iterative sub-contractions

Each clause simplification is a sub-contraction disentangling the tensor network.

Example for definite clauses:

-
-
-



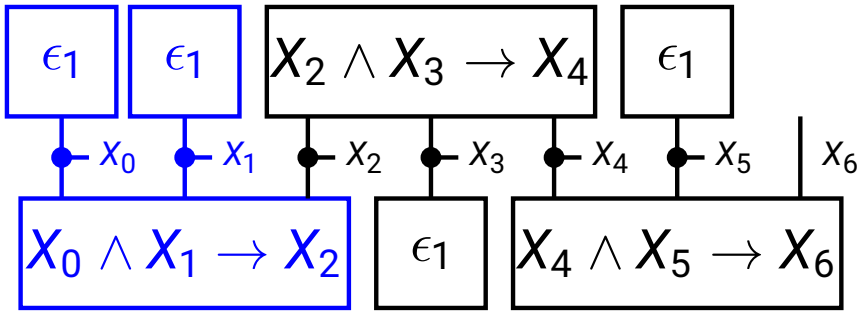
Forward chaining: Disentangling the network

Forward chaining as iterative sub-contractions

Each clause simplification is a sub-contraction disentangling the tensor network.

Example for definite clauses:

- Modus ponens on $X_0 \wedge X_1 \Rightarrow X_2$
-
-



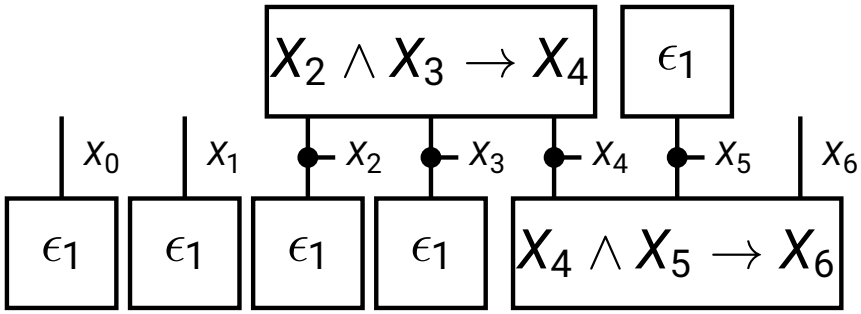
Forward chaining: Disentangling the network

Forward chaining as iterative sub-contractions

Each clause simplification is a sub-contraction disentangling the tensor network.

Example for definite clauses:

- Modus ponens on $X_0 \wedge X_1 \Rightarrow X_2$
-
-



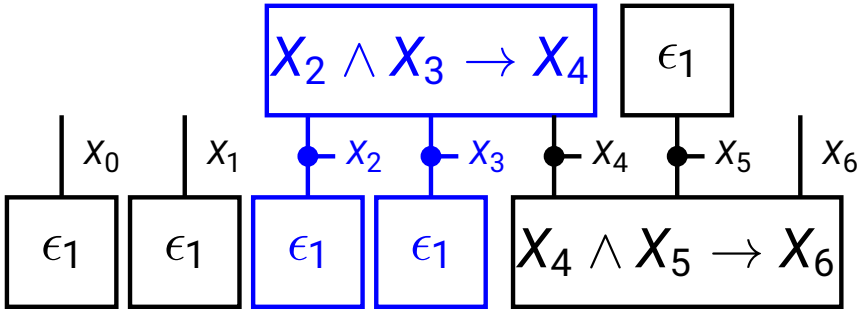
Forward chaining: Disentangling the network

Forward chaining as iterative sub-contractions

Each clause simplification is a sub-contraction disentangling the tensor network.

Example for definite clauses:

- Modus ponens on $X_0 \wedge X_1 \Rightarrow X_2$
- Modus ponens on $X_2 \wedge X_3 \Rightarrow X_4$
-



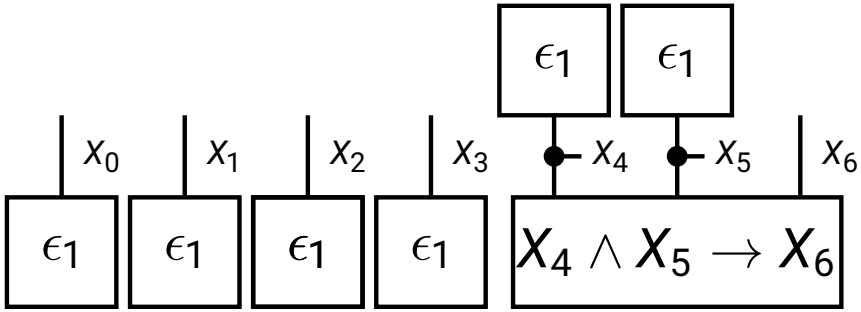
Forward chaining: Disentangling the network

Forward chaining as iterative sub-contractions

Each clause simplification is a sub-contraction disentangling the tensor network.

Example for definite clauses:

- Modus ponens on $X_0 \wedge X_1 \Rightarrow X_2$
- Modus ponens on $X_2 \wedge X_3 \Rightarrow X_4$
-



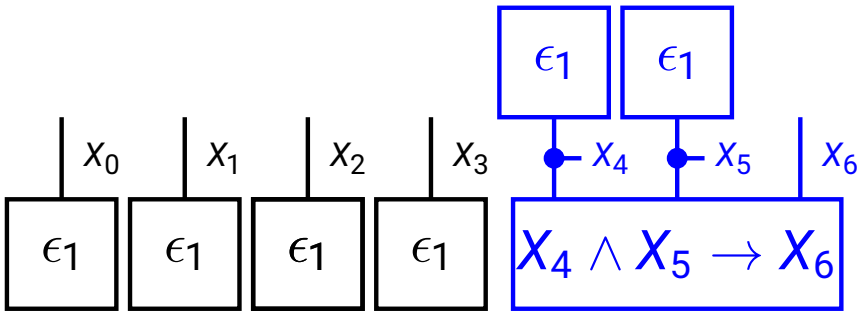
Forward chaining: Disentangling the network

Forward chaining as iterative sub-contractions

Each clause simplification is a sub-contraction disentangling the tensor network.

Example for definite clauses:

- Modus ponens on $X_0 \wedge X_1 \Rightarrow X_2$
- Modus ponens on $X_2 \wedge X_3 \Rightarrow X_4$
- Modus ponens on $X_4 \wedge X_5 \Rightarrow X_6$



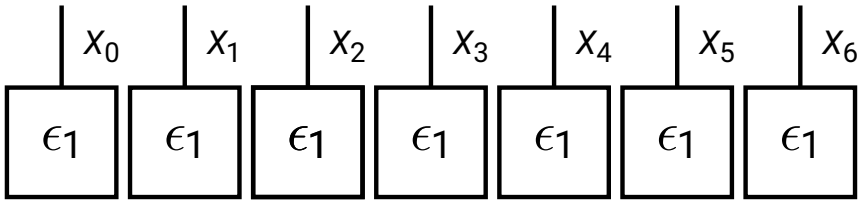
Forward chaining: Disentangling the network

Forward chaining as iterative sub-contractions

Each clause simplification is a sub-contraction disentangling the tensor network.

Example for definite clauses:

- Modus ponens on $X_0 \wedge X_1 \Rightarrow X_2$
- Modus ponens on $X_2 \wedge X_3 \Rightarrow X_4$
- Modus ponens on $X_4 \wedge X_5 \Rightarrow X_6$



Completeness of forward chaining

Forward chaining is **sound**, that is for any clause $\left(\{v\}, y_v\right) \in \mathcal{L}$ in the solution we have

$$\mathcal{KB} \models Z^{\vee}_{\left(\{v\}, y_v\right)} .$$

But: Forward chaining is not always **complete**, that is there might be literals which are entailed by $\mathcal{KB}$ but are not in $\mathcal{L}$.

Completeness of forward chaining

If the knowledge base consists of definite clauses (i.e. exactly one 1 in each $y_{\mathcal{U}}$), completeness of forward chaining can be shown.

Definite Clauses

To show completeness, we need the following property of definite clauses:

Lemma

A knowledge base of definite clauses of length at least 2 always has the states $1_{\mathcal{V}}$ and $0_{\mathcal{V}}$ as models.

Proof sketch: By using the contraction of clauses properties we know that:

- When contracting with $\bigtimes_{v \in \mathcal{V}} \epsilon_1 [X_v]$, all clauses become trivial.
- When contracting with $\bigtimes_{v \in \mathcal{V}} \epsilon_0 [X_v]$, all clauses become trivial.

Thus, $x_{\mathcal{V}} = 1_{\mathcal{V}}$ and $x_{\mathcal{V}} = 0_{\mathcal{V}}$ are models of the knowledge base.

Forward chaining is complete on definite clauses

Theorem

When $\mathcal{KB}$ is a knowledge base of definite clauses, then $\mathcal{L}$ contains all entailed literals.

Sketch of the proof:

- Forward chaining on definite clauses terminates with a knowledge base of definite clauses of length at least 2
- The final knowledge base has $1_{\mathcal{V}}$ and $0_{\mathcal{V}}$ as a model
- The final knowledge base does therefore not entail for any $v \in \mathcal{V}$ the literals X_v or $\neg X_v$

Outline

- Feasibility of contractions
- Monotonicity of Entailment
- Modus ponens and forward chaining
- Constructing inference algorithms

Outlook: Inference algorithms

Inference algorithms focus on subsets of knowledge bases.
We now show that:

- **Soundness:** "Every by a sub knowledge base entailed formula is entailed by the knowledge base itself."
- **Memory:** "Every entailed formula can be added to the knowledge base."

Support tensors

Support tensor to a tensor $\tau\left[X_{[d]}\right]$: $\mathbb{I}_{\neq 0}\left(\tau\left[X_{[d]}\right]\right)$ with coordinates

$$\mathbb{I}_{\neq 0}\left(\tau\left[X_{[d]} = x_{[d]}\right]\right) = \begin{cases} 1 & \text{if} \quad \tau\left[X_0 = x_0, \ldots, X_{d-1} = x_{d-1}\right] \neq 0 \\ 0 & \text{else} \end{cases}$$

Invariance under adding support tensor to sub-contractions

Any support tensor of a sub-contraction can be added to a tensor network: When $\tilde{\mathcal{G}}$ a sub-hypergraph of $\mathcal{G}$, then

$$\left\langle \tau^{\mathcal{G}}\left[X_{[d]}\right]\right\rangle_{[X_{\mathcal{V}}]} = \left\langle \tau^{\mathcal{G}}\left[X_{[d]}\right], \mathbb{I}_{\neq 0}\left(\left\langle \tau^{\tilde{\mathcal{G}}}\left[X_{\mathcal{V}}\right]\right\rangle_{[X_{\mathcal{U}}]}\right)\right\rangle_{[X_{\mathcal{V}}]}$$

In propositional logics, this means:

$$\mathbb{I}_{\neq 0}\left(\left\langle \tau^{\mathcal{G}}\left[X_{[d]}\right]\right\rangle_{[X_{\mathcal{V}}]}\right) \models \mathbb{I}_{\neq 0}\left(\left\langle \tau^{\tilde{\mathcal{G}}}\left[X_{\mathcal{V}}\right]\right\rangle_{[X_{\mathcal{U}}]}\right)$$

Inference algorithms based on sub-contractions

Computation by sub-contractions (including previous messages)

Problem: Many possible sub-contractions

In large tensor networks, there are many possible sub-contractions. For efficiency, different scheduling policies emerge:

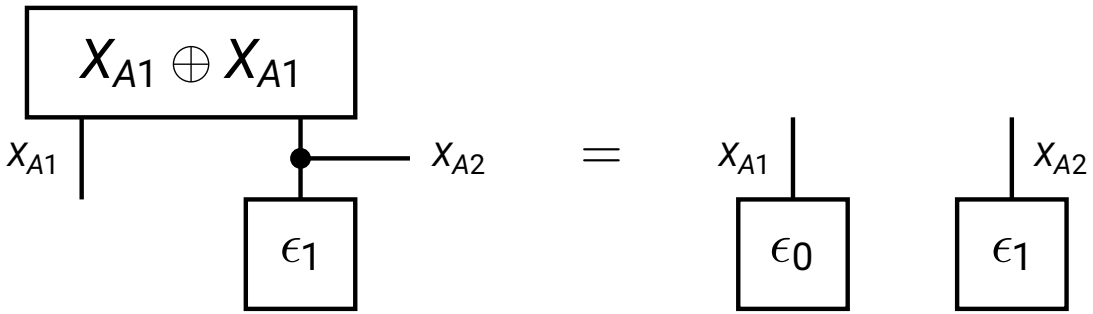
- Syntactical inference algorithms (today)
- Message passing algorithms (next lecture)

Generalization: Contraction perspective

First toy accounting rule $X_{A1} \oplus X_{A2}$ does not have a CNF of definite clauses!

$$X_{A1} \oplus X_{A2} \equiv \left(\neg X_{A1} \vee \neg X_{A2}\right) \wedge \left(X_{A1} \vee X_{A2}\right)$$

Example: When knowing that X_{A2} is true, we get with this accounting that X_{A1} is false:

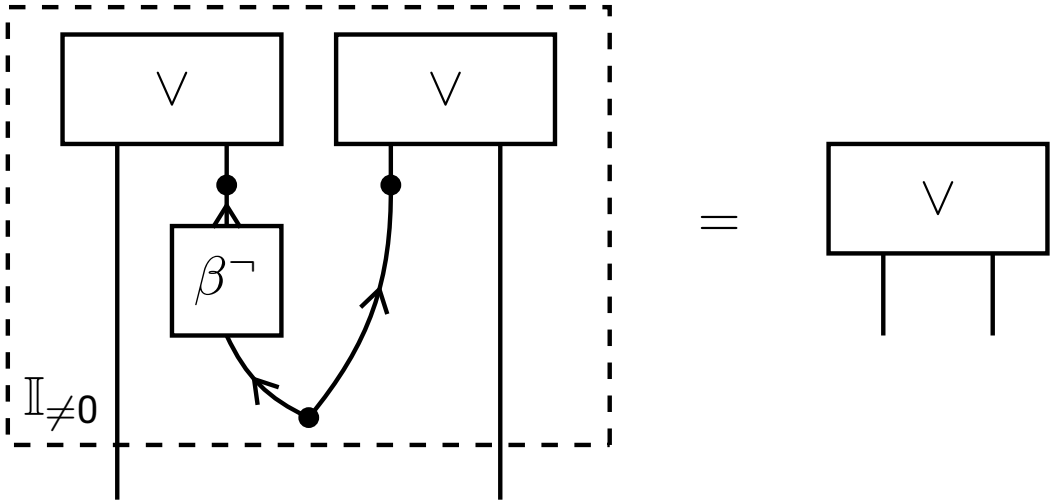


Generalization: Resolution

Resolution Inference Rule

$$\frac{f_0 \vee X\,,\;\; f_1 \vee \neg X}{f_0 \vee f_1}$$

Underlying is the contraction equation:



Scheduling is key

Number of large contractions

The number of possible sub-contractions grows fast with the number k of the subcontraction: $\binom{|\mathcal{E}|}{k}$

Resolution:

- Restricts the possible sub-contractions to those of clauses with complementary clauses
- Requires further scheduling policies for efficiency

Selection of contractions to be performed is central to algorithm design

- Forward chaining
- SLD resolution (e.g. in ProLog)